Understanding Data Types :

A **Data Type** is an instruction to the compiler. It tells the computer two critical things:

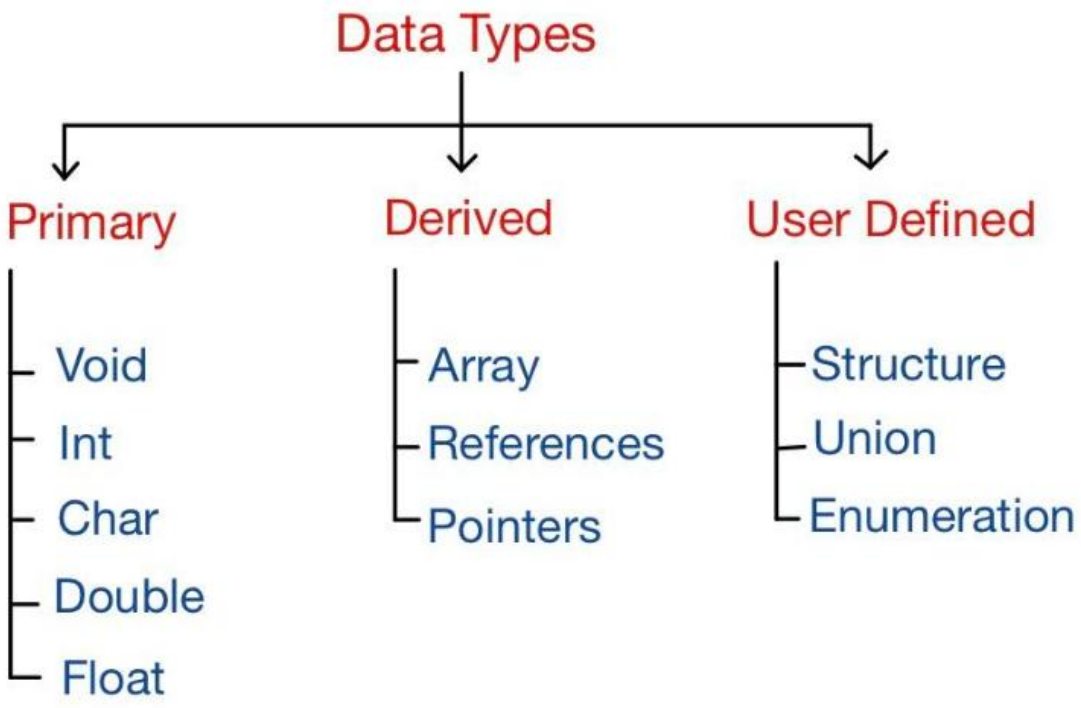
What kind of data is being stored (e.g., text, whole numbers, decimals).

How much memory space to set aside for that data.

Definition: A Data Type specifies the type of value a variable can hold and the exact amount of memory allocated for it.

The C DataType Landscape

In C, data types are broadly categorized into three main families:



1. The Core Data Types

A. Basic (Primary) Data Types

These are the built-in, fundamental building blocks of the language.

Data Type	Used For	Technical Example
char	Single characters, text symbols, or small integers	'A', '$', '7'
int	Whole numbers (no decimals)	42, -105, 0
float	Single-precision decimal numbers	3.14, -0.005
double	Double-precision decimals (more accurate, larger range)	2.7182818284

B. Derived Data Types

Data types that are built by grouping or pointing to basic data types.

Array: A collection of multiple items of the same type stored sequentially.

Pointer: A variable that holds the memory address of another variable.

Function: A block of code that returns a specific data type value.

C. User-Defined Data Types

Custom types created by the programmer to model complex real-world data.

struct (Structure): Groups different data types under one name.

union: Shares the same memory location for different variables.

enum (Enumeration): Assigns names to integer constants.

typedef: Creates a shortcut or alias name for an existing data type.

What is a Qualifier?

A qualifier modifies a data type.

Think of a data type as a vehicle.

int → Car

Now suppose you want:

Small Car
Big Car
Sports Car

You add extra information.

Similarly, in C, we can modify the properties of a datatype by using **qualifiers**.

Example:

```
short int age;
long int population;
unsigned int count;
```

Types of Type Qualifiers

The most important qualifiers are:

1. short
2. long
3. signed
4. unsigned
5. const

1. short

Used when we need a smaller integer.

Example:

```
short int age = 20;
```

or simply

```
short age = 20;
```

Meaning:

Use less memory than normal int.

Real-Life Example

Suppose age can never be:

10 Crore
20 Crore

So why reserve large memory?

A smaller data type is enough.

---

2. long

Used when we need larger values.

Example:

long int population = 1400000000;

Meaning:

Store larger numbers.

Example

Population of India:

1,400,000,000+

A normal small integer may not be enough.

So we use:

long int population;

---

3. signed

Allows both positive and negative values.

Example:

signed int temperature = -10;

Values can be:

-100
-50
0
50
100

Default Behavior

Normally:

int age;

is already signed.

So these are equivalent:

int age;

signed int age;

---

4. unsigned

Allows only positive values.

Example:

unsigned int students = 100;

Valid:

0
10
100
500

Invalid:

-10
-50

Why?

Because the number of students can never be negative.

5. const Qualifier

This is extremely important.

Meaning

Constant = Cannot be changed

Example:

const float PI = 3.14;

Now:

PI = 5.0;

❌ Error

Because PI is constant.

Which Qualifier works with which Data Type?

Data Type	Compatible Qualifiers	Notes
char	signed, unsigned	Used often in systems programming for raw byte manipulation.
int	short, long, signed, unsigned	Highly flexible; can combine them (e.g., unsigned long int).
float	*None*	Sizes are locked by IEEE standards.
double	long	long double provides maximum decimal precision.
void	*None*	Represents the absence of a type.

Memory Sizes in Modern Architectures (32-bit / 64-bit)

Data Type	Typical Modern Size	Memory Visualization
char	1 Byte (8 bits)	[8 bits]
short int	2 Bytes (16 bits)	[16 bits]
int	4 Bytes (32 bits)	[32 bits]
long int	4 or 8 Bytes	Dependent on compiler/OS framework
float	4 Bytes (32 bits)	[32 bits]
double	8 Bytes (64 bits)	[64 bits]
pointer	4 Bytes (32-bit OS) 8 Bytes (64-bit OS)	Scales automatically based on CPU architecture